

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

1 Согласовать вариант задания с преподавателем и внимательно изучить исходные данные к задаче (таблица 6.1).

2 Описать алгоритм работы для решения следующей задачи:

Объявить заданную структуру (по варианту).

Создать динамический массив из S таких структур.

Организовать пользовательское меню из следующих пунктов:

– добавить данные (заполнить одну структуру и добавить в массив);

– очистить данные (удалить массив);

– записать текущий массив в текстовый файл;

– считать данные из файла в массив;

– вывести данные на экран; – выполнить задачи (по варианту); – завершить работу программы.

3 Написать программный код реализации составленного алгоритма с учетом требований и ограничений по индивидуальному заданию.

4 Организовать текстовый пользовательский интерфейс и форматированный вывод данных.

5 Проверить правильность вычислений на тестовых примерах, выполнив серию тестовых прогонов программы.

12	Структура «Проекты»: название проекта, список разработчиков, дата начала, статус проекта (в плане, в работе, отложен, закончен).	
	Задачи:	1. Вывести все проекты со статусом N, отсортировать по убыванию даты начала работы. 2. Найти самого «крутого» разработчика, у которого наибольшее количество проектов в статусе выполнено

ВЫПОЛНЕНИЕ ЗАДАНИЯ

Листинг программы:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <map>
using namespace std;

struct Date {
    int day;
    int month;
    int year;
};

struct Project {
    string name;
    vector<string> developers;
    Date startDate;
    string status;
};

vector<Project> projects;

void addProject() {
    Project project;
    string developer;
    int numDevelopers;

    cout << "Введите название проекта: ";
    cin.ignore();
    getline(cin, project.name);

    cout << "Введите количество разработчиков: ";
    cin >> numDevelopers;
    cin.ignore();

    for (int i = 0; i < numDevelopers; ++i) {
        cout << "Введите имя разработчика " << i + 1 << ": ";
        getline(cin, developer);
        project.developers.push_back(developer);
    }

    cout << "Введите дату начала (день месяц год): ";
    cin >> project.startDate.day >> project.startDate.month >>
project.startDate.year;

    cout << "Введите статус проекта (укажите \"completed\" для завершённого
проекта!): ";
    cin.ignore();
    getline(cin, project.status);

    projects.push_back(project);
}

void clearProjects() {
    projects.clear();
    cout << "Массив проектов очищен.\n";
}
```

```

void displayProjects() {
    for (const auto& project : projects) {
        cout << "Название: " << project.name << '\n';
        cout << "Разработчики: ";
        for (const auto& developer : project.developers) {
            cout << developer << ' ';
        }
        cout << "\nДата начала: " << project.startDate.day << '.' <<
project.startDate.month << '.' << project.startDate.year << '\n';
        cout << "Статус: " << project.status << "\n\n";
    }
}

void displayProjectsByStatus(const string& status) {
    vector<Project> filteredProjects;

    for (const auto& project : projects) {
        if (project.status == status) {
            filteredProjects.push_back(project);
        }
    }

    sort(filteredProjects.begin(), filteredProjects.end(), [](const Project& a,
const Project& b) {
        if (a.startDate.year != b.startDate.year)
            return a.startDate.year < b.startDate.year;
        if (a.startDate.month != b.startDate.month)
            return a.startDate.month < b.startDate.month;
        return a.startDate.day < b.startDate.day;
    });

    for (const auto& project : filteredProjects) {
        cout << "Название: " << project.name << '\n';
        cout << "Разработчики: ";
        for (const auto& developer : project.developers) {
            cout << developer << ' ';
        }
        cout << "\nДата начала: " << project.startDate.day << '.' <<
project.startDate.month << '.' << project.startDate.year << '\n';
        cout << "Статус: " << project.status << "\n";
    }
}

void displayTopDeveloper() {
    map<string, int> developerProjectCount;

    for (const auto& project : projects) {
        if (project.status == "completed") {
            for (const auto& developer : project.developers) {
                developerProjectCount[developer]++;
            }
        }
    }

    auto topDeveloper = max_element(developerProjectCount.begin(),
developerProjectCount.end(),
    [](const pair<string, int>& a, const pair<string, int>& b) {
        return a.second < b.second;
    });

    if (topDeveloper != developerProjectCount.end()) {
        cout << "Разработчик с наибольшим количеством завершённых проектов: " <<
topDeveloper->first
        << " (" << topDeveloper->second << " проектов)\n";
    }
}

```

```

    }
    else {
        cout << "\nНет завершённых проектов.\n";
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    int choice;
    do {
        cout << "Выберите пункт меню\n";
        cout << "1. Добавить новый проект\n";
        cout << "2. Очистить массив проектов\n";
        cout << "3. Вывести все проекты\n";
        cout << "4. Вывести проекты по статусу\n";
        cout << "5. Вывести разработчика с наибольшим количеством завершённых
проектов\n";
        cout << "6. Завершить работу\n";
        cin >> choice;
        cout << "\n";

        switch (choice) {
            case 1:
                addProject();
                break;
            case 2:
                clearProjects();
                break;
            case 3:
                displayProjects();
                break;
            case 4:
            {
                string status;
                cout << "Введите статус: ";
                cin.ignore();
                getline(cin, status);
                displayProjectsByStatus(status);
            }
                break;
            case 5:
                displayTopDeveloper();
                break;
            case 6: {
                cout << "Завершение работы программы.\n";
                exit(0);
                break;
            }
            default:
                cout << "\nНеверный пункт меню.\n\n";
                break;
        }
    } while (choice >= 6);
    return 0;
}

```

Результат выполнения программы (снимок экрана):

```
C:\Users\minsk\source\repos\OAIp\x64\Debug\LR_6.exe

3. Вывести все проекты
4. Вывести проекты по статусу
5. Вывести разработчика с наибольшим количеством завершённых проектов
6. Завершить работу
1

Введите название проекта: asdfghffdsa
Введите количество разработчиков: 4
Введите имя разработчика 1: saddfg
Введите имя разработчика 2: uytre
Введите имя разработчика 3: nmbvc
Введите имя разработчика 4: kjhg
Введите дату начала (день месяц год): 1 12 2023
Введите статус проекта (укажите "completed" для завершённого проекта!): completed
Выберите пункт меню
1. Добавить новый проект
2. Очистить массив проектов
3. Вывести все проекты
4. Вывести проекты по статусу
5. Вывести разработчика с наибольшим количеством завершённых проектов
6. Завершить работу
3

Название: asdfghffdsa
Разработчики: saddfg uytre nmbvc kjhg
Дата начала: 1.12.2023
Статус: completed

Выберите пункт меню
1. Добавить новый проект
2. Очистить массив проектов
3. Вывести все проекты
4. Вывести проекты по статусу
5. Вывести разработчика с наибольшим количеством завершённых проектов
6. Завершить работу
-

```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Составные типы данных. Виды и их назначение.

Составные типы: массивы, перечисления, функции, структуры, ссылки, указатели, объединения и классы.

2 Особенности объявления структур в C++.

Структура в C++ представляет собой производный тип данных, который представляет какую-то определенную сущность, также как и класс. Нередко структуры применительно к C++ также называют классами. И в реальности различия между ними не такие большие. Структура также может определять переменные, функции, конструкторы, деструкторы. Однако обычно структуры служат для хранения каких-то общедоступных данных в виде публичных переменных. Для остальных сценариев используются классы.

3 Обращение (доступ) к членам структуры.

Доступ к полям структуры осуществляется через пользовательские методы, возвращающие тип данных поля.

4 Инициализация структур. Копирование объектов структурного типа.

Структур инициализируются с помощью конструктора. Например:

```
struct person
{
public:
    person(std::string name, unsigned age): name{name}, age{age}
    {
    }
    void print()
    {
        std::cout << "Name: " << name << "\tAge: " << age << std::endl;
    }
private:
    std::string name;
    unsigned age;
};
```

Копирование объекта структуры осуществляется через конструктор копирования, принимающий объект.

5 Определение размера памяти, выделяемой под структуру.

C++ использует методы работы с динамической памятью при помощи операторов `new` и `delete`: `new` — для выделения памяти;; `delete` — для освобождения памяти.

Эти методы применимы и для объектов структур.

6 Тип данных «объединение». Особенности использования объединений.

Объединения используют в двух случаях:

Для создания «универсального» типа данных, способного хранить не единственный, а один из предопределённых типов. Для этого к объединению добавляют целочисленное поле, указывающее тип хранимых в настоящий момент данных:

```
struct variant
{
    union tag_value
    {
        int intValue;
        float floatValue
    } value;
    unsigned storedType;
};
```

Для преобразования между несовместимыми типами. Традиционно для этих целей используют оператор преобразования (T), либо `reinterpret_cast<>`.

Однако эти способы опасны нарушением strict aliasing rule и, как результат, порождением неопределённого (то есть непредсказуемого) поведения.

7 Вложенные структуры. Массивы структур.

Пример вложенных структур

```
struct tag_fio {
    char name[name_length]; /* имя */
    char last[name_length]; /* фамилия */
};
struct tag_person {
    struct tag_fio fio; /* ФИО */
    char sex; /* пол: м или ж */
    unsigned short old; /* возраст */
    char b_date[b_length]; /* дата рождения */
};
```

Пример массива структур

`MyStruct obj1[10];` //Массив из десяти структур. Или просто массив структур.

8 Понятия «файл» и «файловая система».

Файл — именованная область данных на носителе информации, используемая как базовый объект взаимодействия с данными в операционных системах.

Файловая система — порядок, определяющий способ организации, хранения и именования данных на носителях информации в компьютерах, а также в другом электронном оборудовании: цифровых фотоаппаратах, мобильных телефонах и т. п.

9 Классификация файлов. Текстовые и бинарные типы файлов.
В C++ все файлы делятся на два типа: текстовые и бинарные.

10 Понятие «файловый указатель».

Указатель на файл - это указатель на информацию, определяющую различные параметры файла, включая его имя, состояние и текущую позицию. В принципе, указатель на файл идентифицирует конкретный дисковый файл и используется потоком для выполнения операций ввода/вывода. Указатель на файл - это переменная-указатель типа FILE. Чтобы прочитать или записать файлы, программе надо использовать указатели на файлы. Для создания файловой переменной-указателя используется оператор типа:

FILE *fp;

11 Особенности доступа к файлам в C++. Режимы открытия файлов.

При операциях с файлом вначале необходимо открыть файл с помощью функции open(). Данная функция имеет две версии:

open(путь)

open(путь, режим)

Список доступных режимов открытия файла:

- ios::in: файл открывается для ввода (чтения), может быть установлен только для объекта ifstream или fstream;
- ios::out: файл открывается для вывода (записи), при этом старые данные удаляются, может быть установлен только для объекта ofstream или fstream;
- ios::app: файл открывается для дозаписи, старые данные не удаляются;
- ios::ate: после открытия файла перемещает указатель в конец файла;
- ios::trunc: файл усекается при открытии. Может быть установлен, если также установлен режим out;
- ios::binary: файл открывается в бинарном режиме.

12 Основные операции по работе над файлами в C++.

Операции по работе над файлами в C++:

- создание файла
- запись в файл
- чтение с файла
- добавление информации в файл

13. Возможности языка C++ по организации ввода и вывода данных с использованием файлов.

C++ предоставляет мощные средства работы с файлами через классы ifstream, ofstream и fstream, а также поддерживает потоковую обработку данных с помощью операторов << и >>.

14. Способы доступа к данным в файле.

Доступ к данным в файле может быть:

1. Последовательным: чтение или запись данных по порядку.
2. Произвольным: чтение или запись в любое место файла с помощью методов `seekg` и `seekr`.

15. Доступ к структурам из нескольких файлов.

Для доступа к структурам из нескольких файлов используется механизм заголовочных файлов. Определения структур помещаются в заголовочные файлы (.h), а реализация – в файлы исходного кода (.cpp).

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения, реализовано программное средство, удовлетворяющее поставленной задаче, даны ответы на контрольные вопросы.